

LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI *SINGLE LINKED LIST (SLL)*
MENGGUNAKAN BAHASA *JAVA*



Disusun Oleh:

Haliya Isma Husna Putri Ahmadi
2511532002

Dosen Pengampu:
Wahyudi. Dr. S.T.M.T

Asisten Praktikum:
Muhammad Galid

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan Syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas Rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 5 dengan judul “Implementasi *Single Linked List (SLL)* Menggunakan Bahasa *Java*” tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi salah satu tugas dalam mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran bagi penulis dalam memahami konsep struktur data *Single Linked List (SLL)* serta penerapannya dalam pemrograman *Java*. Melalui praktikum ini, penulis mempelajari bagaimana merepresentasikan data dalam bentuk *node* yang saling terhubung menggunakan pointer, serta memahami cara kerja struktur data linear dinamis.

Selain itu, penulis juga mengimplementasikan berbagai operasi dasar pada *Single Linked List*, seperti penambahan data (*insert*) di awal, akhir, dan posisi tertentu, penghapusan data (*delete*) pada berbagai kondisi, penelusuran data (*traversal*), serta pencarian data (*search*). Dengan implementasi ini, penulis dapat memahami alur manipulasi data dalam struktur *linked list* secara lebih mendalam.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Ucapan terima kasih yang sebesar-besarnya penulis sampaikan kepada dosen pengampu, asisten praktikum, serta semua pihak yang terlibat dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca.

Padang, 5 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I.....	5
1.1 Latar Belakang	5
1.2 Tujuan	6
1.3 Manfaat	6
BAB II	7
2.1 Dasar Teori.....	7
2.1.1 Operasi Dasar	8
2.2 Langkah Kerja	9
2.2.1 Pembuatan <i>Package</i> dan <i>Class</i>	9
2.2.2 Membuat Program <i>NodeSLL_2511532002</i>	13
2.2.3 Membuat Program <i>TambahSLL_2511532002</i>	13
2.2.4 Membuat Program <i>PencarianSLL_2511532002</i>	16
2.2.5 Membuat Program <i>HapusSLL_2511532002</i>	16
2.2.6 <i>Upload</i> ke GitHub	20
2.3 Hasil.....	23
2.3.1 Hasil <i>NodeSLL_2511532002</i>	23
2.3.2 Hasil <i>TambahSLL_2511532002</i>	23
2.3.3 Hasil <i>PencarianSLL_2511532002</i>	23
2.3.4 Hasil <i>HapusSLL_2511532002</i>	24
BAB III.....	25
3.1 Kesimpulan	25
3.2 Saran	25
DAFTAR PUSTAKA.....	26
TUGAS PRAKTIKUM PEKAN 3.....	27
1. Kode Program	27
1.1 Kelas <i>Pasien_2511532002</i>	27
1.2 Kelas <i>RumahSakit_2511532002</i>	28
2. Hasil.....	32
3. Kesimpulan	33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya pengelolaan data yang efisien dan terstruktur dalam proses komputasi. Salah satu aspek penting dalam pemrograman adalah penggunaan struktur data yang tepat untuk mengatur dan memanipulasi data sesuai dengan kebutuhan sistem. Salah satu struktur data yang banyak digunakan adalah *Linked List*, khususnya *Single Linked List (SLL)*, yang berperan dalam pengelolaan data secara dinamis.

Single Linked List merupakan struktur data linear yang terdiri dari sekumpulan *node* yang saling terhubung, di mana setiap *node* memiliki dua bagian, yaitu data dan *pointer* (referensi) yang menunjuk ke *node* berikutnya. Berbeda dengan struktur data statis seperti *array*, *Single Linked List* memiliki keunggulan dalam hal fleksibilitas ukuran serta efisiensi dalam operasi penambahan dan penghapusan data.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar *Single Linked List*, cara implementasinya dalam bahasa *Java*, serta penerapan berbagai operasi dasar seperti penambahan (*insert*), penghapusan (*delete*), penelusuran (*traversal*), dan pencarian (*search*). Dengan pemahaman tersebut, penulis diharapkan mampu memilih dan menggunakan struktur data yang tepat untuk menyelesaikan permasalahan secara efektif dan efisien.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

- 1.2.1 Memahami konsep dasar struktur data *Single Linked List (SLL)* sebagai salah satu struktur data linear dinamis.
- 1.2.2 Mengetahui struktur *node* pada *Single Linked List* yang terdiri dari data dan *pointer* ke *node* berikutnya.
- 1.2.3 Mengimplementasikan operasi dasar pada *Single Linked List* menggunakan bahasa *Java*, meliputi penambahan data (*insert*), penghapusan (*delete*), penelusuran (*traversal*), dan pencarian data (*search*).
- 1.2.4 Melatih kemampuan dalam menganalisis alur manipulasi data pada struktur *Single Linked List* untuk menyelesaikan permasalahan pemrograman secara efektif.

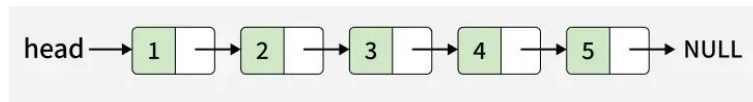
1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum ini antara lain:

- 1.3.1 Menambah pemahaman mengenai penerapan struktur data *Single Linked List* dalam pemrograman *Java*.
- 1.3.2 Membantu mahasiswa memahami cara pengelolaan data secara dinamis tanpa harus menentukan ukuran data di awal.
- 1.3.3 Melatih kemampuan logika pemrograman dalam melakukan operasi penambahan, penghapusan, penelusuran, dan pencarian data.
- 1.3.4 Menjadi dasar pemahaman untuk mempelajari struktur data lain yang lebih kompleks, seperti *Double Linked List*, *Stack*, dan *Queue*.

BAB II PEMBAHASAN

2.1 Dasar Teori



Single Linked List (SLL) merupakan salah satu struktur data linear yang digunakan untuk menyimpan sekumpulan data secara dinamis. Pada struktur ini, setiap elemen disebut sebagai *node*, yang terdiri dari dua bagian utama, yaitu data dan *pointer* (referensi) yang menunjuk ke *node* berikutnya. *Node* pertama disebut sebagai *head*, sedangkan *node* terakhir akan menunjuk ke nilai *null* sebagai penanda akhir dari list [1].

Single Linked List termasuk dalam kategori *Abstract Data Type (ADT)* karena mendefinisikan sekumpulan operasi yang dapat dilakukan tanpa harus bergantung pada detail implementasinya. Beberapa operasi dasar pada *Single Linked List* meliputi penambahan data (*insert*), penghapusan data (*delete*), penelusuran data (*traversal*), dan pencarian data (*search*) [1].

Dibandingkan dengan struktur data statis seperti *array*, *Single Linked List* memiliki keunggulan dalam hal fleksibilitas ukuran karena tidak memerlukan alokasi memori di awal. Selain itu, proses penambahan dan penghapusan data dapat dilakukan dengan lebih efisien, terutama pada bagian tertentu seperti di awal atau di tengah list [1].

Dengan memahami konsep *Single Linked List*, mahasiswa diharapkan mampu mengelola data secara dinamis serta memilih struktur data yang sesuai dalam menyelesaikan permasalahan pemrograman secara efektif dan efisien.

2.1.1 Operasi Dasar

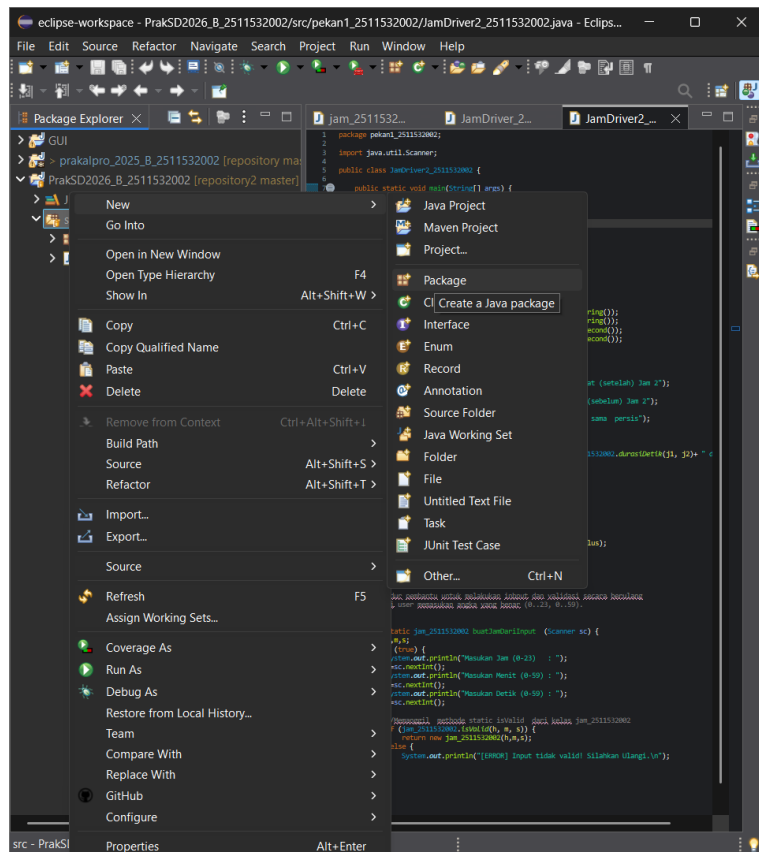
Single Linked List mendukung beberapa operasi dasar, antara lain *Enqueue*

1. *Traversal*: menelusuri seluruh elemen dalam list.
2. *Insertion*: menambahkan node di awal, akhir, atau posisi tertentu.
3. *Deletion*: menghapus node dari awal, akhir, atau posisi tertentu.
4. *Searching*: mencari data tertentu dalam list.

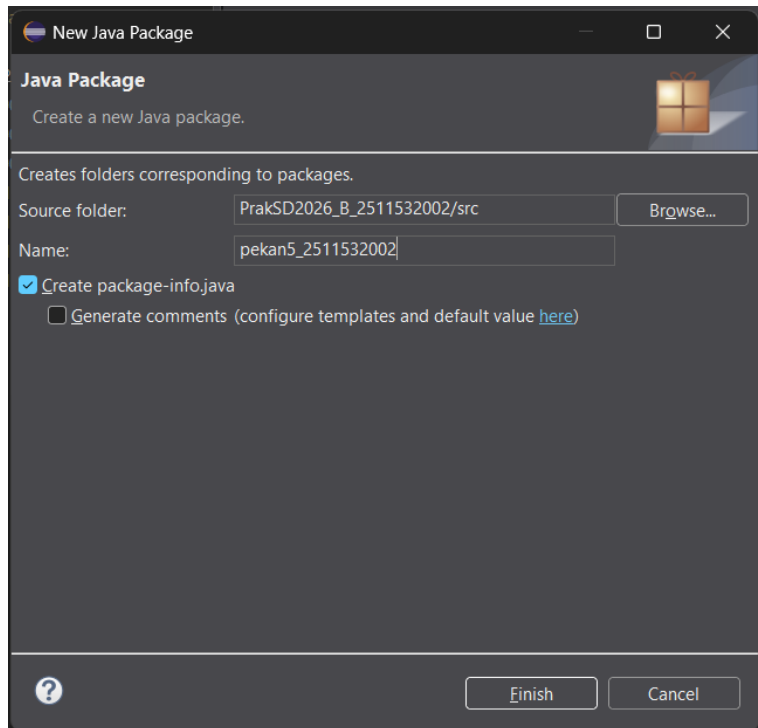
2.2 Langkah Kerja

2.2.1 Pembuatan *Package* dan *Class*

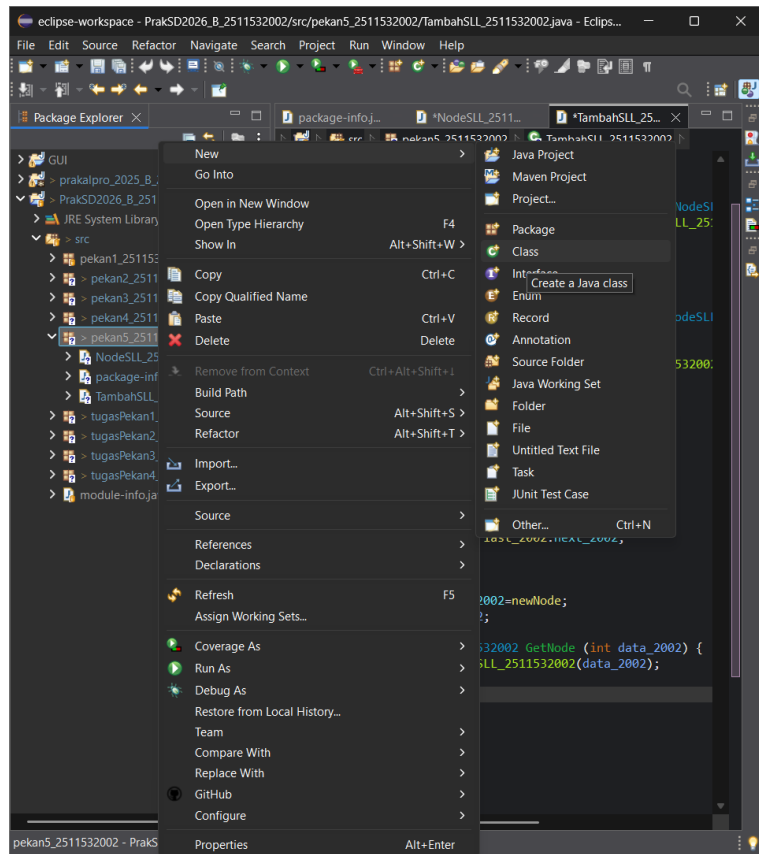
- 1) Sebelum membuat suatu program *user* perlu untuk membuat *package* terlebih dahulu. Klik kanan pada *src* pada *repository* “PrakSD2026_B_2511532002” lalu pilih “New” dan klik “Package”.



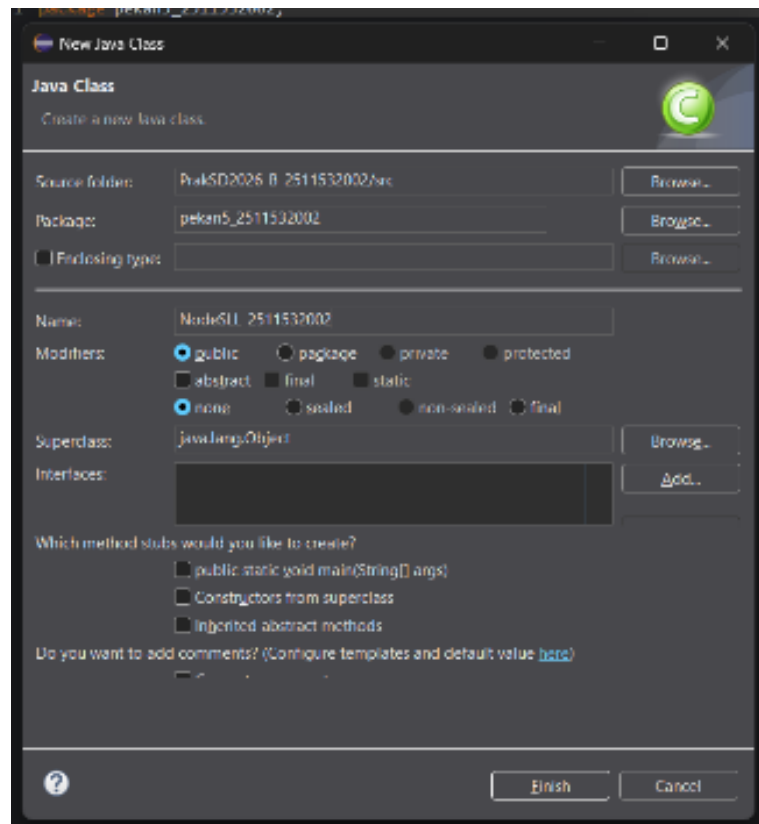
- 2) Setelah itu akan muncul *pop up* “Java Package”, buat nama *package* dengan ketentuan tanpa *capslock*, *space*, dan karakter khusus lainnya. Lalu klik “*finish*”.



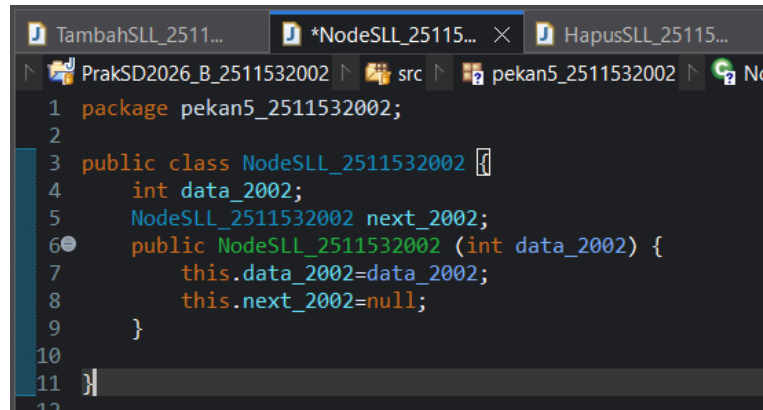
- 3) Selanjutnya adalah membuat membuat *class*. pada package yang telah dibuat tapi klik kanan lalu pilih “New” dan klik “Class”.



- 4) Setelah itu pada menu “*Java Class*” buat nama *class*, dengan ketentuan nama harus capslock diawal kalimat dan tanpa spasi.



2.2.2 Membuat Program *NodeSLL_2511532002*



```

1 package pekan5_2511532002;
2
3 public class NodeSLL_2511532002 {
4     int data_2002;
5     NodeSLL_2511532002 next_2002;
6     public NodeSLL_2511532002 (int data_2002) {
7         this.data_2002=data_2002;
8         this.next_2002=null;
9     }
10
11 }
12

```

Setelah membuat *class QueueArray_2511532002*, maka mahasiswa dapat menulis *syntax* nya.

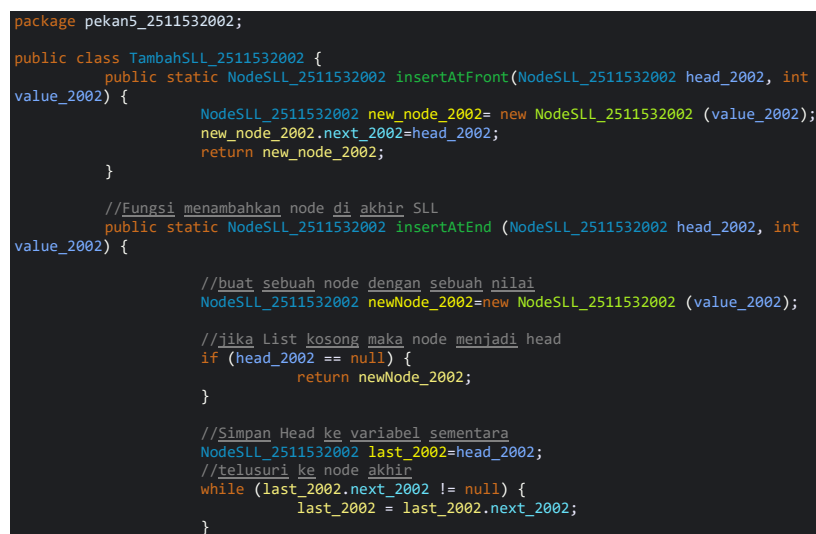
1. Menentukan atribut (*Type*) yang digunakan dalam *Node Single Linked List*.
int data_2002;
NodeSLL_2511532002 next_2002;
2. Membuat konstruktor untuk menginisialisasi *node*.

```

public NodeSLL_2511532002 (int data_2002) {
    this.data_2002 = data_2002;
    this.next_2002 = null;
}

```

2.2.3 Membuat Program *TambahSLL_2511532002*



```

package pekan5_2511532002;

public class TambahSLL_2511532002 {
    public static NodeSLL_2511532002 insertAtFront(NodeSLL_2511532002 head_2002, int value_2002) {
        NodeSLL_2511532002 new_node_2002= new NodeSLL_2511532002 (value_2002);
        new_node_2002.next_2002=head_2002;
        return new_node_2002;
    }

    //Fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511532002 insertAtEnd (NodeSLL_2511532002 head_2002, int value_2002) {
        //buat sebuah node dengan sebuah nilai
        NodeSLL_2511532002 newNode_2002=new NodeSLL_2511532002 (value_2002);

        //jika List kosong maka node menjadi head
        if (head_2002 == null) {
            return newNode_2002;
        }

        //Simpan Head ke variabel sementara
        NodeSLL_2511532002 last_2002=head_2002;
        //telusuri ke node akhir
        while (last_2002.next_2002 != null) {
            last_2002 = last_2002.next_2002;
        }
    }
}

```

```

        //Ubah pointer
        last_2002.next_2002=newNode_2002;
        return head_2002;
    }
    static NodeSLL_2511532002 GetNode (int data_2002) {
        return new NodeSLL_2511532002(data_2002);
    }

    static NodeSLL_2511532002 insertPos (NodeSLL_2511532002 headNode_2002, int
position_2002, int value_2002) {
        NodeSLL_2511532002 head_2002= headNode_2002;

        if (position_2002 <1)
            System.out.println("Invalid Position!");
        if (position_2002 ==1) {
            NodeSLL_2511532002 new_node_2002 = new NodeSLL_2511532002
(value_2002);

            new_node_2002.next_2002=head_2002;
            return new_node_2002;
        }else {
            while (position_2002-- != 0) {
                if (position_2002 == 1) {
                    NodeSLL_2511532002 newNode_2002 = GetNode
(value_2002);

                    newNode_2002.next_2002=headNode_2002.next_2002;
                    headNode_2002.next_2002=newNode_2002;
                    break;
                }
                headNode_2002 =headNode_2002.next_2002;
            }
            if (position_2002 != 1)
                System.out.println("Posisi diluar jangkauan!");
            return head_2002;
        }
    }

    public static void printList(NodeSLL_2511532002 head_2002) {
        NodeSLL_2511532002 curr_2002=head_2002;
        while (curr_2002.next_2002 != null) {
            System.out.print(curr_2002.data_2002+" -->
");

            curr_2002 =curr_2002.next_2002;
        }
        if (curr_2002.next_2002==null) {
            System.out.println(curr_2002.data_2002);
            System.out.println();
        }
    }

    //Main Program
    public static void main(String[] args) {

        //Buat Linked List 2 -> 3 -> 5 -> 6
        NodeSLL_2511532002 head_2002 = new NodeSLL_2511532002(2);
        head_2002.next_2002 = new NodeSLL_2511532002(3);
        head_2002.next_2002.next_2002 = new NodeSLL_2511532002(5);
        head_2002.next_2002.next_2002.next_2002 = new NodeSLL_2511532002(6);

        //Cetak list Asli
        System.out.println("Senerai Berantai awal: ");
        printList(head_2002);

        //Tambahkan Node baru di depan
        System.out.println("tambah 1 simpul di depan: ");
        int data_2002=1;
        head_2002 = insertAtFront (head_2002, data_2002);

        //Cetak Update list
        printList(head_2002);

        //Tambahkan Node baru di belakang
        System.out.println("tambah 1 simpul di belakang: ");
        int data2_2002=7;
        head_2002 =insertAtEnd(head_2002, data2_2002);

        //Cetak Update List
        printList(head_2002);
        System.out.println("tambah 1 simpul ke data 4: ");
        int data3_2002 =4;
        int pos_2002=4;
        head_2002=insertPos (head_2002, pos_2002, data3_2002);

        //Cetak Update List
        printList (head_2002);
    }
}

```

1. Membuat *method* untuk menambahkan *node* di depan (*insertAtFront*).

```

public static NodeSLL_2511532002 insertAtFront(NodeSLL_2511532002
head_2002, int value_2002) {
    NodeSLL_2511532002 new_node_2002 = new
NodeSLL_2511532002(value_2002);
    new_node_2002.next_2002 = head_2002;
    return new_node_2002;
}

```
2. Membuat *method* untuk menambahkan *node* di akhir (*insertAtEnd*).

```

public static NodeSLL_2511532002 insertAtEnd(NodeSLL_2511532002
head_2002, int value_2002)

```
3. Membuat *method* untuk membuat *node* baru (*getNode*).

```

static NodeSLL_2511532002 getNode(int data_2002) {
    return new NodeSLL_2511532002(data_2002);
}

```
4. Membuat *method* untuk menambahkan *node* pada posisi tertentu (*insertPos*).

```

static NodeSLL_2511532002 insertPos(NodeSLL_2511532002
headNode_2002, int position_2002, int value_2002)

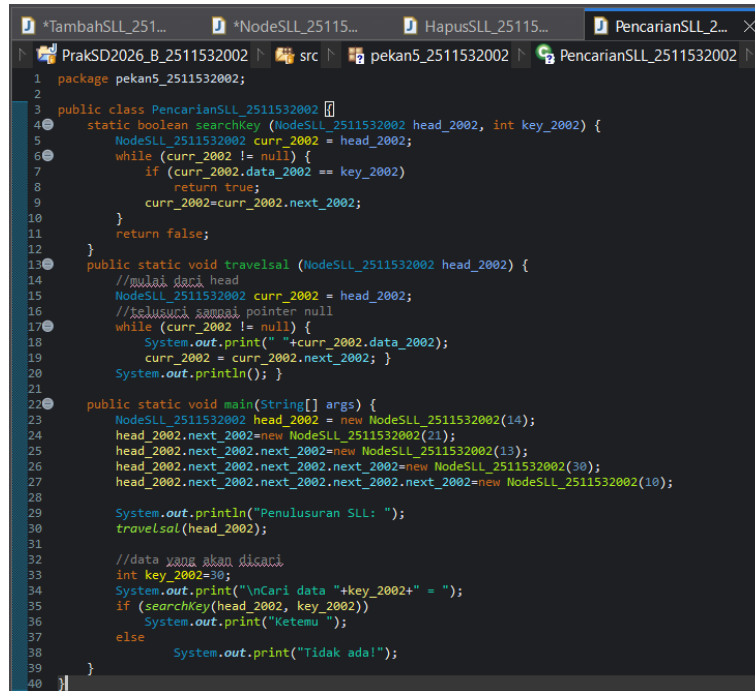
```
5. Membuat *method* untuk menampilkan isi linked list (*printList*).
Membuat main program untuk pengujian.

```

public static void printList(NodeSLL_2511532002 head_2002)

```

2.2.4 Membuat Program *PencarianSLL_2511532002*



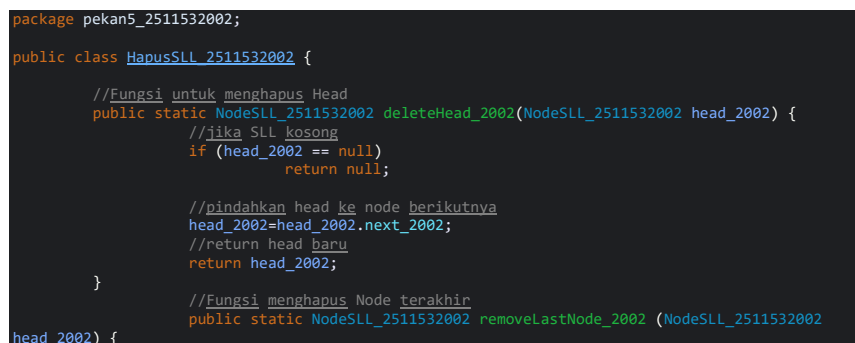
```

1 package pekan5_2511532002;
2
3 public class PencarianSLL_2511532002 {
4     static boolean searchKey (NodeSLL_2511532002 head_2002, int key_2002) {
5         NodeSLL_2511532002 curr_2002 = head_2002;
6         while (curr_2002 != null) {
7             if (curr_2002.data_2002 == key_2002)
8                 return true;
9             curr_2002 = curr_2002.next_2002;
10        }
11        return false;
12    }
13    public static void traversal (NodeSLL_2511532002 head_2002) {
14        //mulai dari head
15        NodeSLL_2511532002 curr_2002 = head_2002;
16        //khususnya pointer null
17        while (curr_2002 != null) {
18            System.out.print(" "+curr_2002.data_2002);
19            curr_2002 = curr_2002.next_2002; }
20        System.out.println(); }
21
22    public static void main(String[] args) {
23        NodeSLL_2511532002 head_2002 = new NodeSLL_2511532002(14);
24        head_2002.next_2002 = new NodeSLL_2511532002(21);
25        head_2002.next_2002.next_2002 = new NodeSLL_2511532002(13);
26        head_2002.next_2002.next_2002.next_2002 = new NodeSLL_2511532002(30);
27        head_2002.next_2002.next_2002.next_2002.next_2002 = new NodeSLL_2511532002(10);
28
29        System.out.println("Penelusuran SLL: ");
30        traversal(head_2002);
31
32        //data yang akan dicari
33        int key_2002=30;
34        System.out.print("\nCari data "+key_2002+" = ");
35        if (searchKey(head_2002, key_2002))
36            System.out.print("Ketemu");
37        else
38            System.out.print("Tidak ada!");
39    }
40 }

```

1. Membuat method untuk mencari data dalam *Single Linked List* (*searchKey*).
static boolean searchKey(NodeSLL_2511532002 head_2002, int key_2002)
2. Membuat method untuk menelusuri isi *linked list* (*traversal*). Method ini digunakan untuk menampilkan seluruh isi linked list. Penelusuran dimulai dari node head hingga mencapai node terakhir (*null*).
public static void traversal(NodeSLL_2511532002 head_2002)
3. Membuat main program untuk pengujian pencarian dan *traversal*. Membuat *linked list* dengan data: 14 -> 21 -> 13 -> 30 -> 10.
public static void main(String[] args)

2.2.5 Membuat Program *HapusSLL_2511532002*



```

package pekan5_2511532002;

public class HapusSLL_2511532002 {

    //Fungsi untuk menghapus Head
    public static NodeSLL_2511532002 deleteHead_2002(NodeSLL_2511532002 head_2002) {
        //jika SLL kosong
        if (head_2002 == null)
            return null;

        //pindahkan head ke node berikutnya
        head_2002 = head_2002.next_2002;
        //return head baru
        return head_2002;
    }

    //Fungsi menghapus Node terakhir
    public static NodeSLL_2511532002 removeLastNode_2002 (NodeSLL_2511532002
head_2002) {

```



```

//jika list kosong, return null
if (head_2002==null) {
    return null;
}
//jika list satu node, hapus node dan return null
if (head_2002.next_2002 == null) {
    return null;
}
//temukan node terakhir ke dua
NodeSLL_2511532002 secondLast_2002 = head_2002;
while (secondLast_2002.next_2002.next_2002 != null) {
    secondLast_2002=secondLast_2002.next_2002;
}
//hapus Node terakhir
secondLast_2002.next_2002=null;
return head_2002;
}
//Fungsi Menghapus Node diposisi tertentu
public static NodeSLL_2511532002 deleteNode_2002 (NodeSLL_2511532002
head_2002, int position_2002) {
    NodeSLL_2511532002 temp_2002=head_2002;
    NodeSLL_2511532002 prev_2002=null;

    //jika LinkedList NULL
    if (temp_2002 == null)
        return head_2002;

    //case 1: Head di hapus
    if (position_2002==1) {
        head_2002=temp_2002.next_2002;
        return head_2002;
    }

    //case 2: menghapus node ditengah
    // telusuri dari node yang dihapus
    for (int i=1; temp_2002 != null && i <position_2002;
i++) {
        prev_2002 = temp_2002;
        temp_2002 = temp_2002.next_2002;
    }
    //jika ditemukan hapus node
    if(temp_2002 != null) {
        prev_2002.next_2002=temp_2002.next_2002;
    }else {
        System.out.println("Data Tidak
Ada!");
    }
    return head_2002;
}

// Fungsi Cetak SLL
public static void printList (NodeSLL_2511532002 head_2002) {
    NodeSLL_2511532002 curr_2002=head_2002;
    while (curr_2002.next_2002 != null) {
        System.out.print(curr_2002.data_2002+" -->
");
        curr_2002=curr_2002.next_2002;
        if (curr_2002.next_2002 == null) {
            System.out.println
(curr_2002.data_2002);
        }
    }
    System.out.println();
}

//Kelas Main
public static void main(String[] args) {
    //Buat SLL 1 -> 2 -> 3-> 4 -> 5 -> 6 ->
    NodeSLL_2511532002 head_2002 = new
NodeSLL_2511532002(1);
    head_2002.next_2002=new
NodeSLL_2511532002(2);
    head_2002.next_2002.next_2002=new
NodeSLL_2511532002(3);
    head_2002.next_2002.next_2002.next_2002=new
NodeSLL_2511532002(4);
    head_2002.next_2002.next_2002.next_2002.next_2002=new NodeSLL_2511532002(5);
    head_2002.next_2002.next_2002.next_2002.next_2002.next_2002=new NodeSLL_2511532002(6);

    //cetak List Awal
    System.out.println("List awal: ");
    printList(head_2002);

    //Hapus Node
    head_2002 = deleteHead_2002 (head_2002);
    System.out.println("List setelah Head
dihapus: ");
    printList(head_2002);

    //Hapus Node Terakhir

```

```

        terakhir dihapus: ");
        position_2002);
        dihapus: ");
    }
}

head_2002 = removeLastNode_2002(head_2002);
System.out.println("List setelah Simpul");
printList(head_2002);

//Deleting Node Posisi 2
int position_2002 = 2;
head_2002 = deleteNode_2002(head_2002,
//Print List After Deletion
System.out.println("List setelah posisi 2
printList(head_2002);

```

1. Membuat Membuat method untuk menghapus *node* pertama (*deleteHead*). Jika *linked list* kosong (*head == null*), maka langsung mengembalikan *null*. Jika tidak, *head* dipindahkan ke *node* berikutnya (*head.next*).

```

public static NodeSLL_2511532002
deleteHead_2002(NodeSLL_2511532002 head_2002)

```

2. Membuat *method* untuk menghapus *node* terakhir (*removeLastNode*).

```

public static NodeSLL_2511532002
removeLastNode_2002(NodeSLL_2511532002 head_2002)

```

3. Membuat method untuk menghapus *node* pada posisi tertentu (*deleteNode*). Dengan case tertentu mulai dari *head* dan ditengah

//case 1: Head di hapus

```

if (position_2002==1) {
    head_2002=temp_2002.next_2002;
    return head_2002;
}

```

//case 2: menghapus node ditengah

// telusuri dari node yang dihapus

```

for (int i=1; temp_2002 != null && i < position_2002; i++) {
    prev_2002 = temp_2002;
    temp_2002 = temp_2002.next_2002;
}

```

//Jika ditemukan hapus node

```

if(temp_2002 != null) {

```

```

    prev_2002.next_2002=temp_2002.next_2002;
} else {

```

```

System.out.println("Data Tidak Ada!");
}
return head_2002;

```

4. Membuat method untuk menampilkan hasil

```
public static void printList (NodeSLL_2511532002 head_2002) {

    NodeSLL_2511532002 curr_2002 = head_2002;
    while (curr_2002.next_2002 != null) {

        System.out.print(curr_2002.data_2002 + " --> ");
        curr_2002 = curr_2002.next_2002;
        if (curr_2002.next_2002 == null) {
            System.out.println (curr_2002.data_2002);

            System.out.println();
        }
    }
}
```

5. Membuat main rogram untuk pengujian operasi penghapusan.

```
public static void main(String[] args) {
    //Buat SLL 1 -> 2 -> 3-> 4 -> 5 -> 6 -> NULL
    NodeSLL_2511532002 head_2002 = new NodeSLL_2511532002(1);
    head_2002.next_2002 = new NodeSLL_2511532002(2);
    head_2002.next_2002.next_2002 = new NodeSLL_2511532002(3);
    head_2002.next_2002.next_2002.next_2002 = new
    NodeSLL_2511532002(4);
    head_2002.next_2002.next_2002.next_2002.next_2002 = new
    NodeSLL_2511532002(5);
    head_2002.next_2002.next_2002.next_2002.next_2002.next_200
    2 = new NodeSLL_2511532002(6);

    //cetak List Awal
    System.out.println("List awal: ")
    printList(head_2002);

    //Hapus Node
    head_2002 = deleteHead_2002 (head_2002);
    System.out.println("List setelah Head dihapus: ");
    printList(head_2002);

    //Hapus Node Terakhir
    head_2002 = removeLastNode_2002(head_2002);
    System.out.println("List setelah Simpul terakhir dihapus: ");
    printList(head_2002);

    //Deleting Node Posisi 2
    int position_2002 = 2;
    head_2002 = deleteNode_2002(head_2002, position_2002);
```

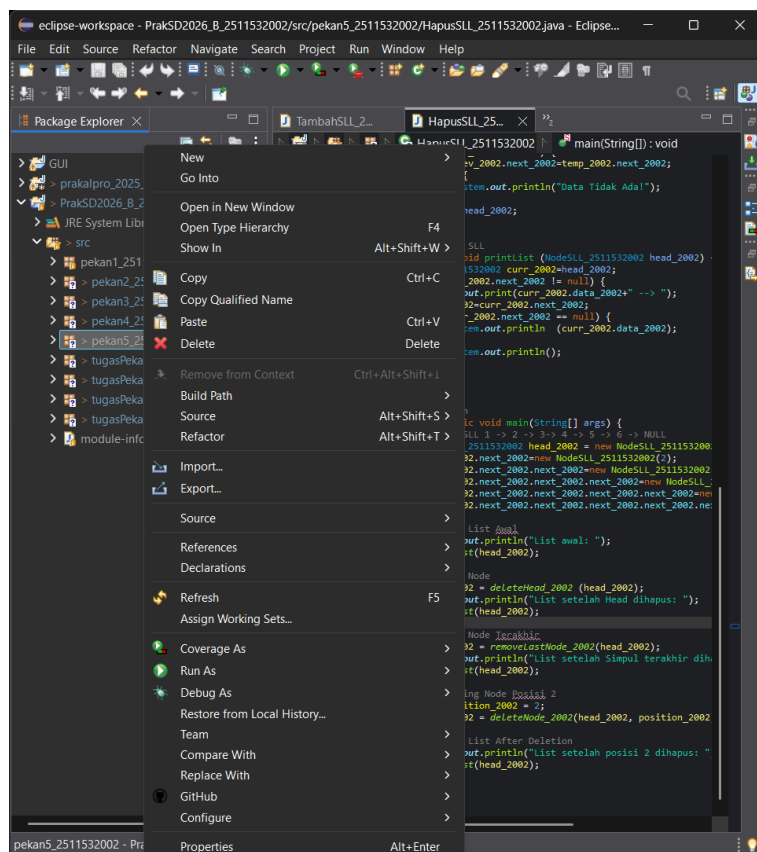
```
//Print List After Deletion
System.out.println("List setelah posisi 2 dihapus: ");
printList(head_2002);
}

}
```

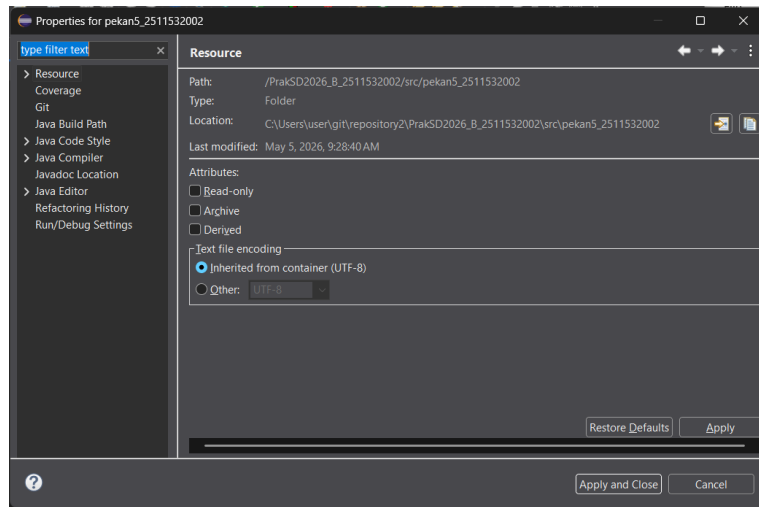
2.2.6 Upload ke GitHub

1. Setelah semua program selesai dibuat, selanjutnya adalah *user* perlu untuk memasukan program yang dibuat di Eclipse ke *repository* GitHub. Selain dengan cara “*Team*” atau “*Commit and Push*”.

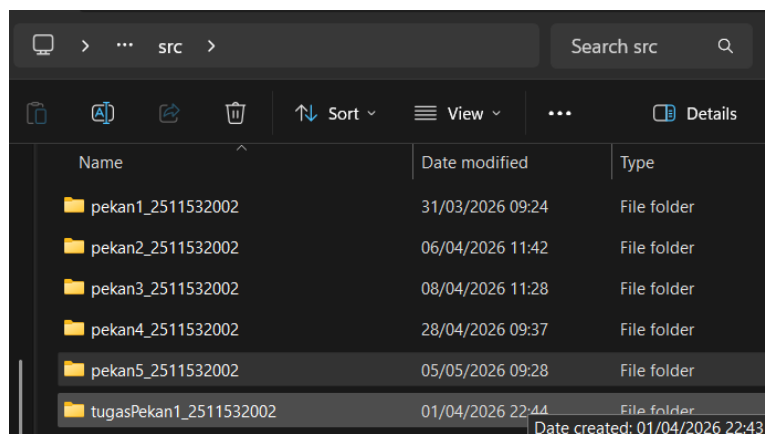
Bisa juga dengan cara *upload* manual. Caranya klik kanan pada *package*, lalu pilih *properties*.



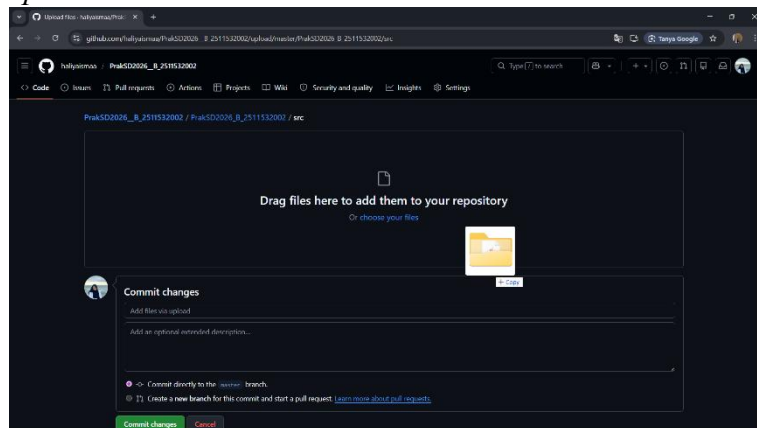
2. Maka akan muncul *pop up menu* “*Properteis for pekan5_2511532002*”. Lalu klik *icon* “*Show in System Folder*”.



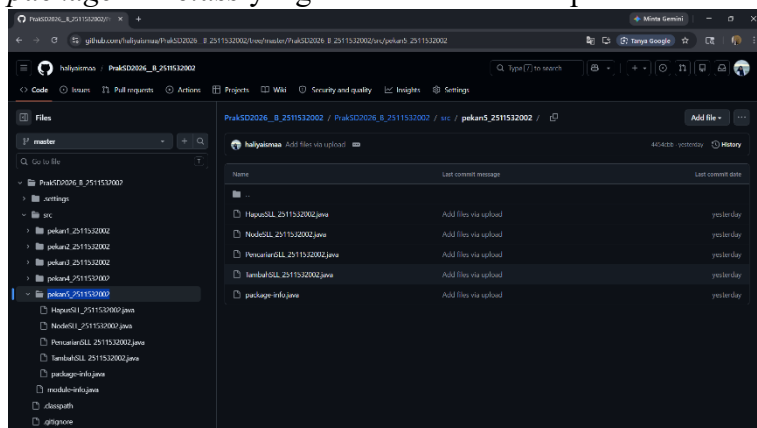
3. Maka akan menunjukan lokasi *file* “*pekan4_2511532002*” pada *File Explorer computer*.



4. Setelah itu *drag and drop file* “*pekan4_2511532002*” ke GitHub. Lalu tambahkan keterangan jika perlu. Setelah itu klik *upload*.

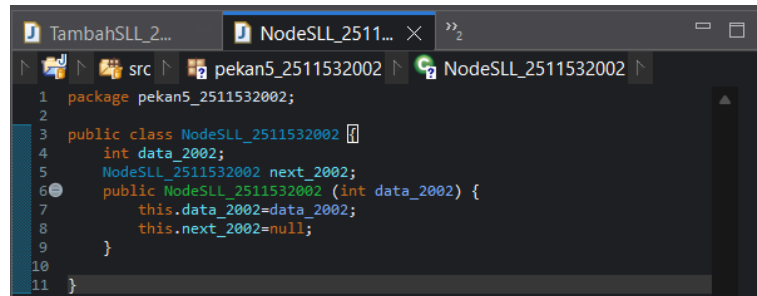


5. Maka dapat dilihat pada *repository GitHub* bahwa file “*pekan5_2511532002*” telah terunggah via *Upload*. Dapat kita lihat disini jika kita buka, maka akan muncul *file package* dan *class* yang telah di buat di Eclipse.



2.3 Hasil

2.3.1 Hasil *NodeSLL_2511532002*

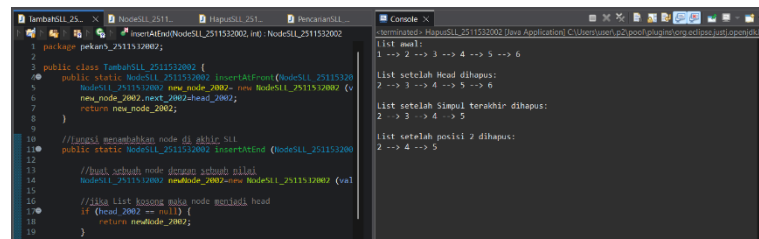


```

1 package pekan5_2511532002;
2
3 public class NodeSLL_2511532002 {
4     int data_2002;
5     NodeSLL_2511532002 next_2002;
6     public NodeSLL_2511532002 (int data_2002) {
7         this.data_2002=data_2002;
8         this.next_2002=null;
9     }
10
11 }

```

2.3.2 Hasil *TambahSLL_2511532002*



```

1 package pekan5_2511532002;
2
3 public class TambahSLL_2511532002 {
4     public static NodeSLL_2511532002 insertAtFront(NodeSLL_2511532002 head, int data_2002) {
5         NodeSLL_2511532002 new_node_2002= new NodeSLL_2511532002 (data_2002);
6         new_node_2002.next_2002=head;
7         return new_node_2002;
8     }
9
10    //fungsi menambahkan node di public SLL
11    public static NodeSLL_2511532002 insertAtEnd (NodeSLL_2511532002 head, int data_2002) {
12        //buat sebuah node dengan sebuah nilai
13        NodeSLL_2511532002 new_node_2002= new NodeSLL_2511532002 (data_2002);
14        //jika list kosong maka node menjadi head
15        if (head == null) {
16            return new_node_2002;
17        }
18    }
19
20 }

```

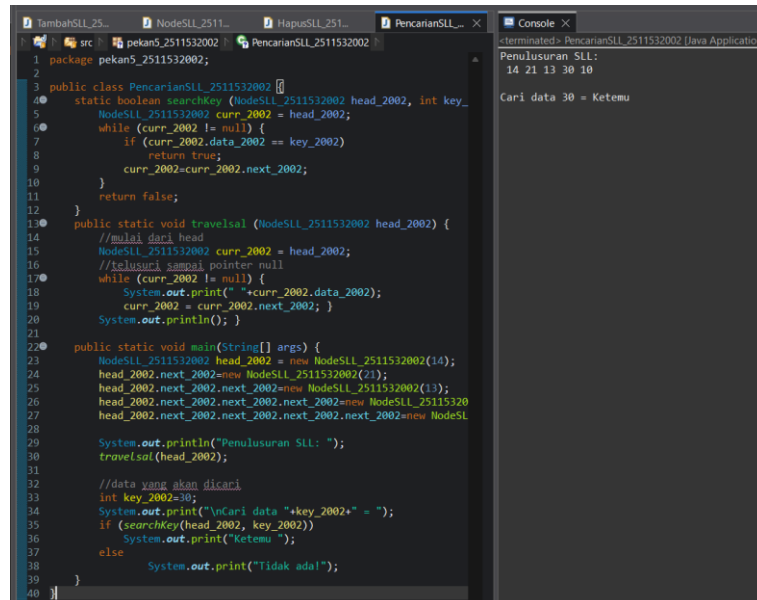
Console Output:

```

<terminated> HapusSLL_2511532002 [Java Application] C:\Users\user\appdata\local\programs\java\jre\bin\java.exe
List awal:
1 --> 2 --> 3 --> 4 --> 5 --> 6
List setelah Head dihapus:
2 --> 3 --> 4 --> 5 --> 6
List setelah Simpul terakhir dihapus:
2 --> 3 --> 4 --> 5
List setelah posisi 2 dihapus:
2 --> 4 --> 5

```

2.3.3 Hasil *PencarianSLL_2511532002*



```

1 package pekan5_2511532002;
2
3 public class PencarianSLL_2511532002 {
4     static boolean searchKey (NodeSLL_2511532002 head_2002, int key_2002) {
5         NodeSLL_2511532002 curr_2002 = head_2002;
6         while (curr_2002 != null) {
7             if (curr_2002.data_2002 == key_2002) {
8                 return true;
9             }
10            curr_2002=curr_2002.next_2002;
11        }
12        return false;
13    }
14
15    public static void travelSLL (NodeSLL_2511532002 head_2002) {
16        //mulai dari head
17        NodeSLL_2511532002 curr_2002 = head_2002;
18        //telusuri sampai pointer null
19        while (curr_2002 != null) {
20            System.out.print(" "+curr_2002.data_2002);
21            curr_2002 = curr_2002.next_2002; }
22        System.out.println();
23    }
24
25    public static void main(String[] args) {
26        NodeSLL_2511532002 head_2002 = new NodeSLL_2511532002(14);
27        head_2002.next_2002=new NodeSLL_2511532002(21);
28        head_2002.next_2002.next_2002=new NodeSLL_2511532002(13);
29        head_2002.next_2002.next_2002.next_2002=new NodeSLL_2511532002(30);
30        head_2002.next_2002.next_2002.next_2002.next_2002=new NodeSLL_2511532002(10);
31
32        System.out.println("Penelusuran SLL: ");
33        travelSLL(head_2002);
34
35        //data yang akan dicari
36        int key_2002=30;
37        System.out.print("\nCari data "+key_2002+" = ");
38        if (searchKey(head_2002, key_2002)) {
39            System.out.print("Ketemu ");
40        } else {
41            System.out.print("Tidak ada!");
42        }
43    }
44 }

```

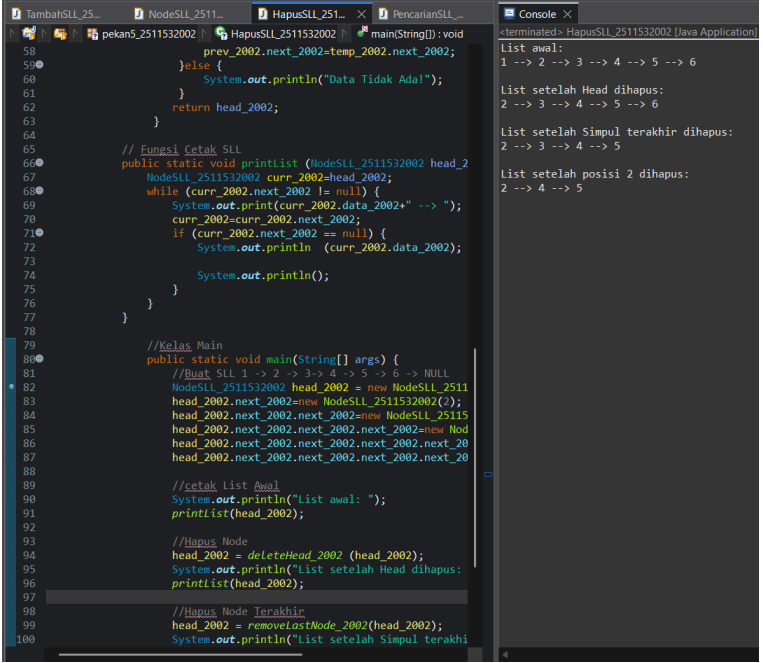
Console Output:

```

<terminated> PencarianSLL_2511532002 [Java Application]
Penelusuran SLL:
14 21 13 30 10
Cari data 30 = Ketemu

```

2.3.4 Hasil *HapusSLL_2511532002*



```

58         prev_2002.next_2002=temp_2002.next_2002;
59     }else {
60         System.out.println("Data Tidak Adal");
61     }
62     return head_2002;
63 }
64
65 // Fungsi Cetak SLL
66 public static void printList (NodeSLL_2511532002 head_2
67 NodeSLL_2511532002 curr_2002=head_2002;
68 while (curr_2002.next_2002 != null) {
69     System.out.print(curr_2002.data_2002+" --> ");
70     curr_2002=curr_2002.next_2002;
71     if (curr_2002.next_2002 == null) {
72         System.out.println (curr_2002.data_2002);
73     }
74     System.out.println();
75 }
76 }
77
78 //Kelas Main
79 public static void main(String[] args) {
80     //Buat SLL 1 -> 2 -> 3-> 4 -> 5 -> 6 -> NULL
81     NodeSLL_2511532002 head_2002 = new NodeSLL_2511
82     head_2002.next_2002=new NodeSLL_2511532002(2);
83     head_2002.next_2002.next_2002=new NodeSLL_25115
84     head_2002.next_2002.next_2002.next_2002=new Nod
85     head_2002.next_2002.next_2002.next_2002.next_20
86     head_2002.next_2002.next_2002.next_2002.next_20
87
88     //cetak List Awal
89     System.out.println("List awal: ");
90     printList(head_2002);
91
92     //Hapus Node
93     head_2002 = deleteHead_2002 (head_2002);
94     System.out.println("List setelah Head dihapus:
95     printList(head_2002);
96
97     //Hapus Node Terakhir
98     head_2002 = removeLastNode_2002(head_2002);
99     System.out.println("List setelah Simpul terakhir
100

```

<terminated> HapusSLL_2511532002 [Java Application]
 List awal:
 1 --> 2 --> 3 --> 4 --> 5 --> 6
 List setelah Head dihapus:
 2 --> 3 --> 4 --> 5 --> 6
 List setelah Simpul terakhir dihapus:
 2 --> 3 --> 4 --> 5
 List setelah posisi 2 dihapus:
 2 --> 4 --> 5

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Single Linked List (SLL)* merupakan salah satu struktur data linear yang digunakan untuk menyimpan data secara dinamis menggunakan *node* yang saling terhubung melalui pointer. Setiap *node* terdiri dari data dan referensi ke node berikutnya, sehingga memungkinkan pengelolaan data yang fleksibel.

Dalam praktikum ini, *Single Linked List* berhasil diimplementasikan menggunakan bahasa pemrograman *Java* dengan membuat class *node* serta berbagai operasi dasar. Operasi yang telah dilakukan meliputi penambahan data (*insert*) di awal, akhir, dan posisi tertentu, penghapusan data (*delete*) pada berbagai kondisi, penelusuran data (*traversal*), serta pencarian data (*search*). Seluruh operasi tersebut dapat berjalan dengan baik sesuai konsep struktur data yang ada.

Melalui praktikum ini, penulis dapat memahami cara kerja *Single Linked List*, khususnya dalam mengelola data secara dinamis serta melakukan manipulasi data dengan lebih efisien dibandingkan struktur data statis seperti array. Dengan demikian, *Single Linked List* menjadi salah satu struktur data penting dalam pemrograman untuk menyelesaikan berbagai permasalahan secara efektif.

3.2 Saran

Saran untuk praktikum selanjutnya adalah agar mahasiswa lebih teliti dalam memahami penulisan *syntax* pemrograman *Java*. Selain itu, diperlukan ketelitian dalam penulisan *syntax* pemrograman *Java*, terutama dalam penggunaan pointer (referensi) antar node agar tidak terjadi kesalahan seperti null pointer atau kesalahan logika program.

DAFTAR PUSTAKA

[1] *GeeksforGeeks*, “*Singly Linked List Tutorial*,” 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/singly-linked-list-tutorial/>

Diakses pada: 6 Mei 2026.

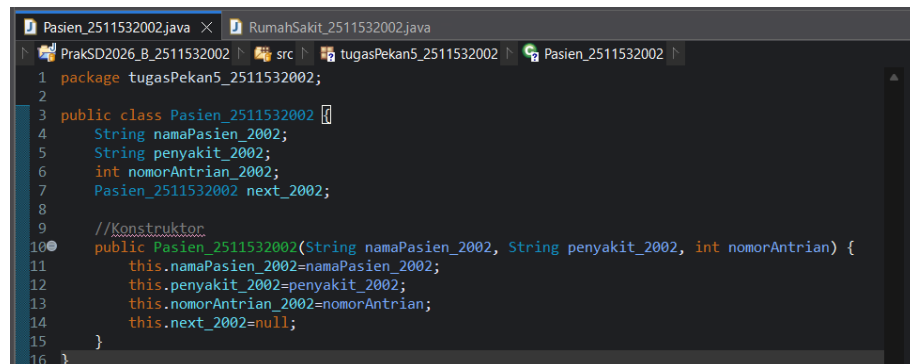
[2] *GeeksforGeeks*, “*Linked List Data Structure*,” 2025. [Online]. Available: <https://www.geeksforgeeks.org/data-structures/linked-list/>

Diakses pada: 6 Mei 2026.

TUGAS PRAKTIKUM PEKAN 5

1. Kode Program

1.1 Kelas *Pasien_2511532002*



```
1 package tugasPekan5_2511532002;
2
3 public class Pasien_2511532002 {
4     String namaPasien_2002;
5     String penyakit_2002;
6     int nomorAntrian_2002;
7     Pasien_2511532002 next_2002;
8
9     //Konstruktor
10    public Pasien_2511532002(String namaPasien_2002, String penyakit_2002, int nomorAntrian) {
11        this.namaPasien_2002=namaPasien_2002;
12        this.penakit_2002=penyakit_2002;
13        this.nomorAntrian_2002=nomorAntrian;
14        this.next_2002=null;
15    }
16 }
```

Penjelasan :

1) *package tugasPekan5_2511532002;*
Menjelaskan posisi file berada pada package bernama “tugaspekan5_2511532002”.

2) *public class Pasien_2511532002 {*
Mendefinisikan *class* bernama *Pasien_2511532002* yang digunakan sebagai *node*, pada struktur data *single linked list*.

3) *Atribut*

- *String namaPasien_2002;*
- *String penyakit_2002;*
- *Int nomorAntrian_2002;*
- *Pasien_2511532002 next_2002;*

Merupakan atribut dari *class Pasien_2511532002* yang digunakan untuk menyimpan data seperti.

4) *Konstruktor*

```
public Pasien_2511532002(String namaPasien_2002,
String penyakit_2002, int nomorAntrian) {
    this.namaPasien_2002=namaPasien_2002;
    this.penakit_2002=penyakit_2002;
    this.nomorAntrian_2002=nomorAntrian;
    this.next_2002=null;
}
```

Merupakan konstruktor yang digunakan untuk mengisialisasikan nilai atribut.

1.2 Kelas RumahSakit_2511532002

```
package tugasPekan5_2511532002;

import java.util.Scanner;

public class RumahSakit_2511532002 {
    Pasien_2511532002 head_2002;
    int counter_2002=0;

    //insert
    void insertPasien_2002(String nama_2002, String penyakit_2002) {
        counter_2002++;
        Pasien_2511532002 newNode_2002 = new Pasien_2511532002(nama_2002,
        penyakit_2002, counter_2002);

        if (head_2002 == null) {
            head_2002 = newNode_2002;
        } else {
            Pasien_2511532002 temp_2002 = head_2002;
            while (temp_2002.next_2002 != null) {
                temp_2002 = temp_2002.next_2002;
            }
            temp_2002.next_2002 = newNode_2002;
        }
        System.out.println("Pasien Berhasil Didafarkan!");
        System.out.println("Nomor Antrian: " + counter_2002);
    }

    //delete head
    void deleteHead_2002() {
        if (head_2002 == null) {
            System.out.println("Antrian kosong!");
            return;
        }

        System.out.println("Memanggil: " + head_2002.namaPasien_2002 +
        " | Keluhan: " + head_2002.penyakit_2002);

        head_2002 = head_2002.next_2002;
    }

    //display
    void display_2002() {
        if (head_2002 == null) {
            System.out.println("Antrian kosong!");
            return;
        }

        Pasien_2511532002 temp_2002 = head_2002;
        System.out.println("\n=== DAFTAR ANTRIAN ===");

        while (temp_2002 != null) {
            System.out.println("No: " + temp_2002.nomorAntrian_2002 +
            " | Nama: " + temp_2002.namaPasien_2002 +
            " | Keluhan: " + temp_2002.penyakit_2002);
            temp_2002 = temp_2002.next_2002;
        }
    }

    //search
    void search_2002(String namaCari_2002) {
        Pasien_2511532002 temp_2002 = head_2002;
        boolean ketemu_2002 = false;
    }
}
```

```

        while (temp_2002 != null) {
            if (temp_2002.namaPasien_2002.equalsIgnoreCase(namaCari_2002)) {
                System.out.println("Pasien ditemukan:");
                System.out.println("No: " + temp_2002.nomorAntrian_2002 +
                    " | Nama: " + temp_2002.namaPasien_2002 +
                    " | Keluhan: " + temp_2002.penyakit_2002);
                ketemu_2002 = true;
                break;
            }
            temp_2002 = temp_2002.next_2002;
        }

        if (!ketemu_2002) {
            System.out.println("Pasien tidak ditemukan!");
        }
    }

    //status
    void status_2002() {
        if (head_2002 == null) {
            System.out.println("Antrian kosong!");
            return;
        }

        int jumlah_2002 = 0;
        Pasien_2511532002 temp_2002 = head_2002;

        while (temp_2002 != null) {
            jumlah_2002++;
            temp_2002 = temp_2002.next_2002;
        }

        System.out.println("Total Pasien: " + jumlah_2002);
        System.out.println("Pasien terdepan: " + head_2002.namaPasien_2002);
    }

    //Main Program
    public static void main(String[] args) {
        Scanner input_2002=new Scanner(System.in);
        RumahSakit_2511532002 rs_2002=new RumahSakit_2511532002();

        int pilihan_2002=0;

        System.out.println("\n ===== ANTRIAN RUMAH SAKIT NIM: 2511532002
=====");
        System.out.println("1. Daftarkan Pasien (Insert)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");

        while (pilihan_2002 !=6) {

            System.out.print("\n Pilihan: ");

            pilihan_2002=input_2002.nextInt();
            input_2002.nextLine();

            switch (pilihan_2002) {
                case 1:
                    System.out.print("Masukkan Nama Pasien: ");
                    String nama_2002 =input_2002.nextLine();
                    System.out.print("Masukkan Keluhan: ");
                    String keluhan_2002=input_2002.nextLine();
                    rs_2002.insertPasien_2002(nama_2002,
keluhan_2002);

                    break;

                case 2:

```

```

        rs_2002.deleteHead_2002();
        break;

        case 3:
            rs_2002.display_2002();
            break;

        case 4:
            System.out.print("Cari nama pasien: ");
            String cari_2002 = input_2002.nextLine();
            rs_2002.search_2002(cari_2002);
            break;

        case 5:
            rs_2002.status_2002();
            break;

        case 6:
            System.out.println("Program selesai.");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
    }
}

input_2002.close();
}

```

Penjelasan:

- 1) *package tugasPekan5_2511532002;*
Menjelaskan posisi file berada pada package bernama “tugaspekan5_2511532002”.
- 2) *public class RumahSakit_2511532002 {*
Merupakan *class* utama (*driver*) yang digunakan untuk menjalankan program.
- 3) *void insertPasien_2002(String nama_2002, String penyakit_2002) {*
Merupakan *method* untuk menambahkan pasien baru ke dalam antrian Data pasien akan dimasukkan pada bagian akhir linked list sehingga sesuai dengan konsep *FIFO (First In First Out)*.
- 4) *void deleteHead_2002()*
Digunakan untuk memanggil atau menghapus pasien terdepan (*head*) dari antrian.
- 5) *Void display_2002()*
Digunakan untuk menampilkan seluruh data pasien yang terdapat dalam antrian.

- 6) *search_2002(String namaCari_2002)*
Digunakan untuk mencari data pasien berdasarkan nama pasien.
- 7) *void status_2002()*
Digunakan untuk menampilkan status antrian seperti jumlah pasien dan pasien terdepan.
- 8) *public static void main(String[] args)Tampilkan Antrian");*
Merupakan program utama yang digunakan untuk menjalankan sistem antrian rumah sakit. Yang menampilkan menu program dan dengan perulangan *while* untuk memilih menu.

2. Hasil

```
Console X
<terminated> RumahSakit_2511532002 [Java Application] C:\Users\

===== ANTRIAN RUMAH SAKIT NIM: 2511532002 =====
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 1
Masukkan Nama Pasien: Haliya
Masukkan Keluhan: Demam
Pasien Berhasil Didaftarkan!
Nomor Antrian: 1

Pilihan: 1
Masukkan Nama Pasien: Ian
Masukkan Keluhan: Asam Lambung
Pasien Berhasil Didaftarkan!
Nomor Antrian: 2

Pilihan: 3

=== DAFTAR ANTRIAN ===
No: 1 | Nama: Haliya | Keluhan: Demam
No: 2 | Nama: Ian | Keluhan: Asam Lambung

Pilihan: 2
Memanggil: Haliya | Keluhan: Demam

Pilihan: 3

=== DAFTAR ANTRIAN ===
No: 2 | Nama: Ian | Keluhan: Asam Lambung

Pilihan: 4
Cari nama pasien: Ian
Pasien ditemukan:
No: 2 | Nama: Ian | Keluhan: Asam Lambung

Pilihan: 5
Total Pasien: 1
Pasien terdepan: Ian

Pilihan: 6
Program selesai.
```


3. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Singly Linked List (SLL) dapat digunakan untuk mengelola data secara dinamis dan terstruktur. Dalam implementasi yang telah dibuat, SLL digunakan untuk merepresentasikan sistem antrian pasien di rumah sakit dengan memanfaatkan konsep FIFO (First In First Out), di mana pasien yang datang terlebih dahulu akan dilayani lebih dahulu.

Melalui program yang dikembangkan, beberapa operasi dasar pada SLL berhasil diimplementasikan, seperti penambahan data (insert) di bagian akhir, penghapusan data di bagian depan (delete head), penelusuran data (traversal/display), serta pencarian data (search). Selain itu, program juga mampu menampilkan informasi jumlah pasien dan kondisi antrian saat ini.

Dari praktikum ini, dapat dipahami bahwa penggunaan SLL memberikan fleksibilitas dalam pengelolaan data karena tidak memerlukan alokasi memori yang tetap seperti array. Dengan demikian, struktur data ini sangat cocok digunakan dalam kasus yang membutuhkan penambahan dan penghapusan data secara dinamis, seperti sistem antrian.